

Pivoting: tuneles a la red interna

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

Pivotar es usar una maquina ya comprometida como puente para llegar a hosts internos que tu atacante no alcanza directamente. Se monta un tunel (SOCKS) a traves del pivot y se enruta el trafico de tus herramientas por el, con lo que puedes escanear y atacar la red interna como si estuvieras dentro.

Las redes se segmentan: el pivot tiene una segunda interfaz o rutas a rangos que tu no ves. Un tunel reverse (chisel, ligolo-ng) hace que el pivot conecte hacia ti, esquivando firewalls que bloquean entrada. Por un SOCKS solo pasa TCP, de ahí que en nmap uses -sT y no SYN scan.

Modelo mental

Entrada controlada -> transformaciones -> validación -> componente final
-> sink -> efecto observable -> decisión

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

Cuando ya tienes acceso a una maquina y descubres una red interna nueva (ip a, ip route, arp -a muestran rangos que tu Kali no tiene). Cada host interno alcanzado reinicia el ciclo recon/enum/acceso.

Cómo identificarla

- El pivot tiene una IP en un rango distinto al que tu atacas (ej. 10.10.20.x).
- ip route o arp -a revelan hosts internos que no salían en tu escaneo inicial.
- Servicios internos que solo responden a traves del proxy.

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

Levanta el tunel y prueba un curl/nc contra un puerto conocido de la red interna antes de escanear.

Evidencia esperada: El puerto SOCKS local escuchando (ss -tlnp) y respuesta del host interno a traves del proxy.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.
3. Define la primitiva mínima indicada por la fuente.

4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
ssh -D 1080 -N user@$IP
```

Objetivo y contexto: Si tienes SSH al pivot, -D monta un proxy SOCKS local sin instalar nada. Es la vía más rápida y limpia con credenciales SSH válidas.

Resultado esperado: No imprime nada (-N): el éxito es el puerto 1080 escuchando en tu Kali (compruébalo con `ss -tlnp`). A partir de ahí, todo va con `proxychains`.

Capa 2: prueba documentada

```
proxychains nmap -sT -Pn -n -p80,445 10.10.20.5
```

Objetivo y contexto: `proxychains` rutea `nmap` por el SOCKS hacia la red interna. -sT (connect) es obligatorio: por un SOCKS no pasa el SYN scan y darías todo por cerrado.

Resultado esperado: Puertos abiertos del host interno. Ve a pocos puertos (es lento por el túnel). Servicios abiertos = una mini-room nueva dentro de la red.

Anatomía de los payloads

La primera prueba es `ssh -D 1080 -N user@$IP`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

El puerto SOCKS local escuchando (`ss -tlnp`) y respuesta del host interno a través del proxy.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Confirma la red interna desde el pivot (`ip a`, `ip route`, `arp -a`).
- Monta el túnel: `ssh -D` si tienes SSH, o `chisel`/`ligolo` en reverse si solo ejecutas binarios.
- Enruta tus herramientas por el SOCKS con `proxychains` (`nmap` con `-sT`).
- Valida con un objetivo pequeño y repite el ciclo de ataque contra los hosts internos.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- `ip route/arp -a` no muestran ninguna red distinta a la que ya atacas: no hay nada que pivotar.
- No tienes forma de ejecutar binarios en el pivot (solo lectura) y tampoco tienes SSH con -D disponible.

- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

La maquina comprometida ve una red que tu Kali no alcanza; la usas de puente montando un tunel. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y preconditiones.
2. Identificar entrada, componente y permisos.
3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.

5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook](#) (pivoting-tunnel)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.